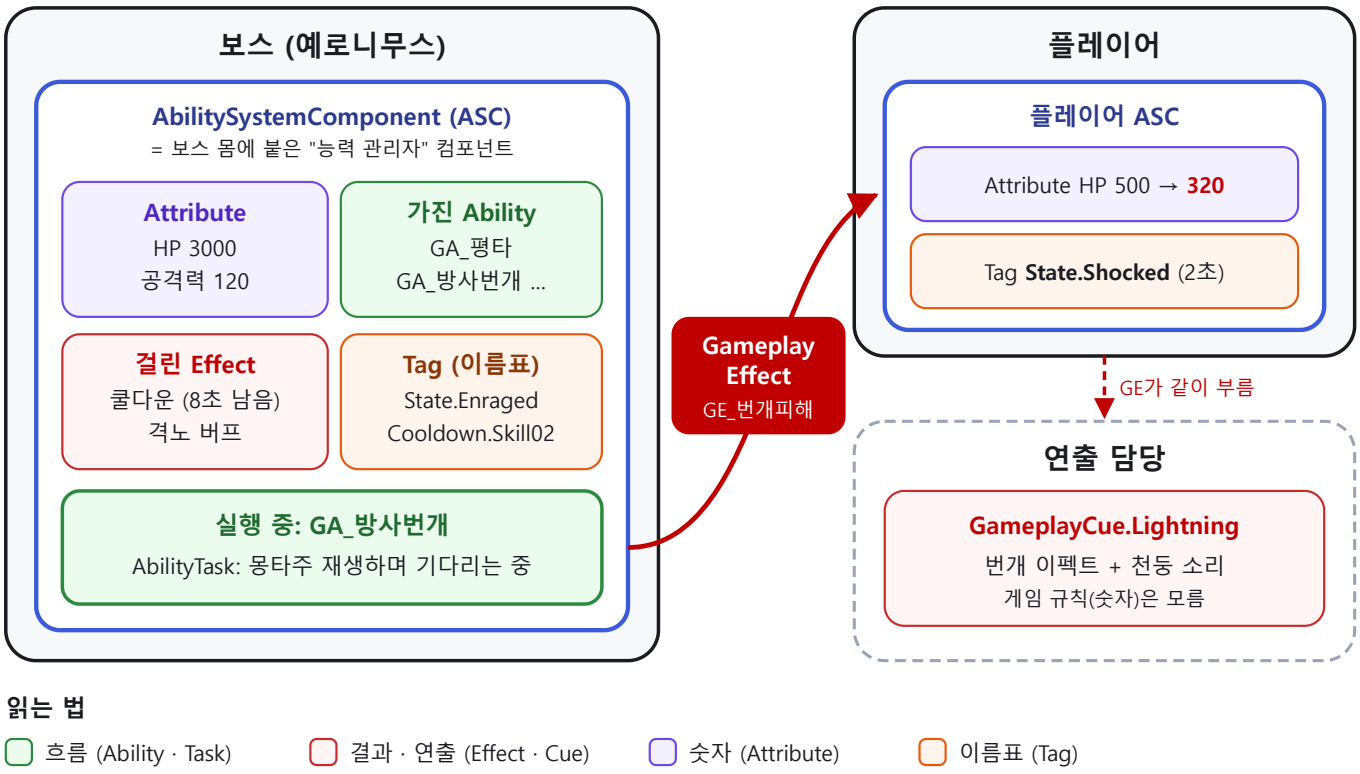
BOSS · GAS BASICS

GAS 쉽게 보기 — 보스 "방사형 번개"로 배우는 용어

GAS(Gameplay Ability System) = 언리얼이 제공하는 "기술 · 스탯 · 효과"를 다루는 틀.
한 문장으로: **기술(Ability)이 흐름을 짜고, 맞으면 효과(Effect)가 숫자를 바꾸고, 연출(Cue)이 보여 준다.**



용어 10개를 네 묶음으로

1. **주인과 숫자**
   - ASC · Attribute · Tag — 캐릭터가 "무엇을 들고 있나"
2. **기술과 기다림**
   - GameplayAbility · AbilityTask · 쿨다운/코스트 — "무엇을 어떤 순서로 하나"
3. **신호와 위치**
   - AnimNotify · GameplayEvent · Targeting — "언제, 어디에"
4. **결과와 연출**
   - GameplayEffect · GameplayCue — "맞으면 어떻게 되고, 어떻게 보이나"

1 주인과 숫자

1. AbilitySystemComponent (ASC)

- **뜻** 캐릭터에 붙이는 컴포넌트 하나. 아래 것들을 전부 들고 있는 "능력 관리자"
- **들고 있는 것** 가진 Ability 목록 · Attribute · 지금 걸린 Effect · Tag
- **보스 예시** 보스 프리팹에 ASC 하나 → "방사형 번개 써"도, "번개 맞았다"도 전부 ASC를 거침

2. Attribute (AttributeSet)

- **뜻** 숫자로 된 스탯. HP · 공격력 · 방어력처럼 Effect가 바꾸는 대상
- **보스 예시** 보스 공격력 120 → 번개 피해 계산에 쓰임 / 플레이어 HP 500 → 번개 맞고 320

3. GameplayTag

- **뜻** 점(.)으로 이어 붙인 이름표. 문자열처럼 보이지만 미리 등록해 두고 씀
- **쓰는 곳 셋** 상태 표시(State.Stunned) · 이벤트 이름(Event.Boss.Strike) · 조건(Cooldown.Skill02 있으면 못 씀)
- **보스 예시** 보스에 State.Stunned 가 붙으면 → "이 태그 있으면 기술 금지" 규칙에 걸려 패턴이 끊김

ASC = 그릇, Attribute = 숫자, Tag = 붙였다 뗐다 하는 이름표

2 기술과 기다림

1. GameplayAbility (GA)

- **뜻** 기술 하나의 "대본". 시작(Activate)부터 끝(End)까지 무엇을 어떤 순서로 할지 적은 코드/블루프린트
- **보스 예시** GA_방사번개 = "내려찍기 재생 → 내려찍는 순간 원 위 12곳에 번개 → 애니 끝나면 종료"
- **끊기** Cancel 되면 안에서 돌던 기다림(Task)까지 한 번에 정리됨

2. AbilityTask

- **뜻** Ability 안에서 "끝날 때까지 기다리는 일" 하나. 끝나면 Ability에 알려 줌
- **대표** PlayMontageAndWait(애니 재생하고 끝까지) · WaitDelay(N초) · WaitGameplayEvent(신호 올 때까지)
- **보스 예시** 내려찍기 몽타주를 틀어 놓고, 동시에 "Strike 신호 오면 깨워 줘"를 걸어 둠

3. 쿨다운 · 코스트

- **뜻** "쓸 수 있나"를 판정하는 문. GAS에선 둘 다 Effect로 만듦
- **보스 예시** 쿨다운 Effect가 보스에 Cooldown.Skill02 태그를 8초 붙임 → 그동안 GA_방사번개는 시작 자체가 막힘
- **주의** GAS는 "쓸 수 있나"만 판정 — 여러 기술 중 **무엇을 쓸지 고르는 건 AI(BT)의 몫**

Ability = 대본, Task = 대본 속 "이 장면 끝날 때까지"

3 신호와 위치

1. AnimNotify

- **뜻** 애니메이션의 특정 프레임에 꽂아 두는 표시. 그 프레임이 재생되면 코드가 불림
- **보스 예시** 내려찍기 몽타주의 "지팡이가 땅에 닿는 프레임"에 Notify 하나

2. GameplayEvent

- **뜻** "태그 이름 + 약간의 데이터"를 캐릭터에게 보내는 메시지
- **보스 예시** Notify가 보스에게 Event.Boss.Strike 를 보냄 → 기다리던 WaitGameplayEvent Task가 깨어남
- **핵심** 애니는 **이름만 보내고 누가 받는지 모름** → 같은 애니를 다른 기술이 다른 일로 재사용 가능

3. Targeting (TargetData · Targeting System)

- **뜻** "어디에 / 누구에게"를 정하는 부품. 모양(구 · 박스 · 원기둥) 고르기 → 거르기 → 정렬
- **보스 예시** 플레이어를 중심으로 반지름 5m 원 위 12개 점 → 각 점 반경 1m 안의 플레이어

Notify = "지금!" 이라는 큐 사인, Event = 그 사인을 이름으로 전하는 무전

4 결과와 연출

1. GameplayEffect (GE)

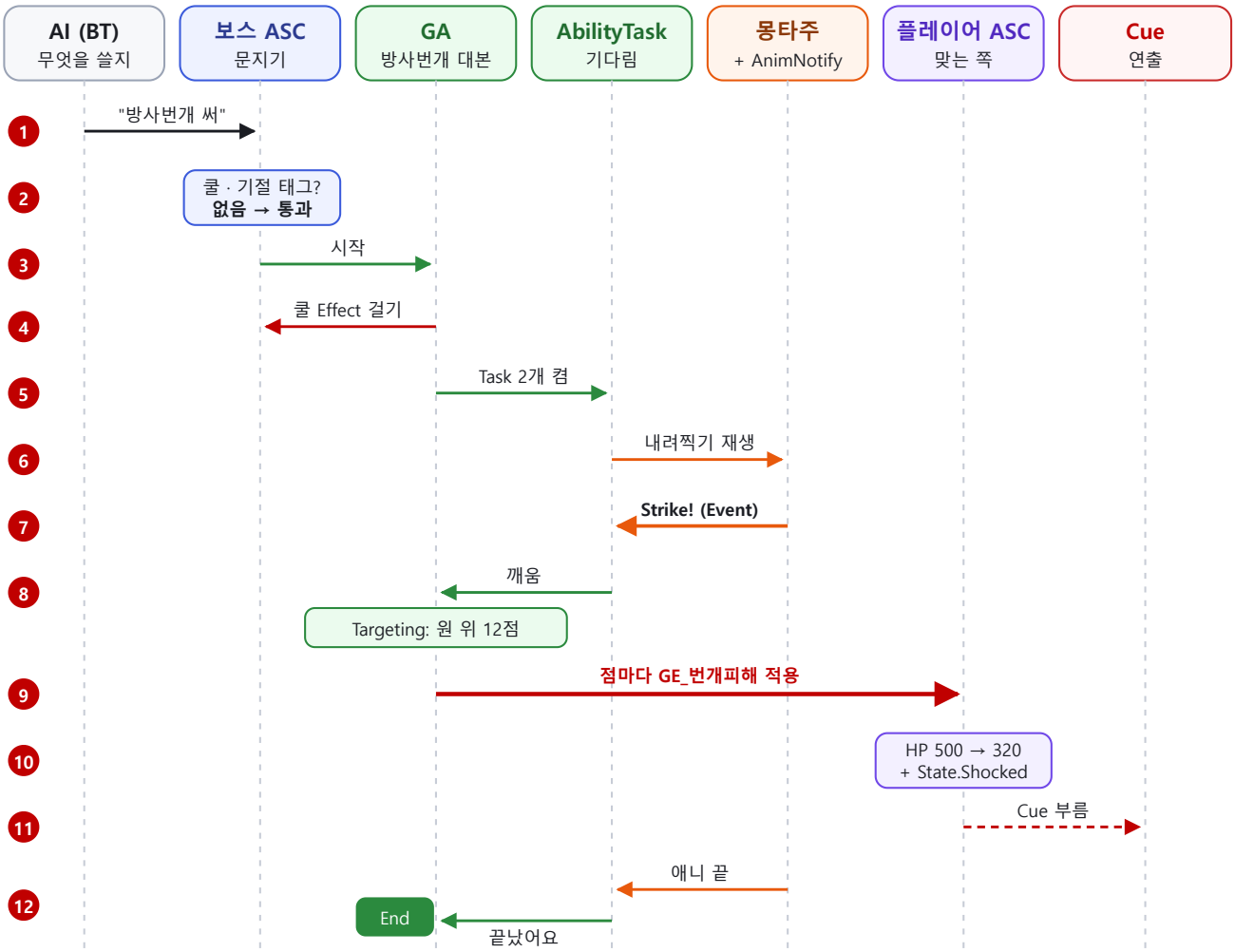
- **뜻** "맞으면 어떻게 되나"를 적은 **데이터 시트**. 코드가 아니라 설정값 묶음
- **길이** Instant(즉시 한 번: HP −180) · Duration(10초간: 공격력 −20%) · Infinite(뗄 때까지)
- **안에 적는 것** 어떤 Attribute를 얼마나(Modifier) · 복잡한 계산식(Execution) · 붙일 태그 · 같이 부를 Cue
- **보스 예시** GE_번개피해 = Instant, HP −(보스 공격력 × 1.5), 태그 State.Shocked 2초, Cue.Lightning
- **Spec** 쓸 때마다 GE를 "이번 값"(시전자 공격력 등)으로 찍어 낸 실행본 — 그래서 같은 GE를 여러 보스가 공유 가능

2. GameplayCue

- **뜻** 이펙트 · 소리만 담당하는 연출 부품. GameplayCue.○○ 태그로 불림
- **보스 예시** GE_번개피해가 적용될 때 Cue.Lightning 이 그 위치에 번개 이펙트 + 천둥 소리
- **핵심** Cue는 숫자를 모르고, GE는 이펙트를 모름 → 연출만 바꿔도 규칙은 그대로

GE = 심판의 판정표, Cue = 조명 · 음향팀

5 방사형 번개 한 번 — 누가 누구에게 무엇을



1. 쓸 수 있나 (1~2)

- AI가 고르고, 보스 ASC가 태그로 막을지 판정 — Cooldown.Skill02나 State.Stunned 가 있으면 여기서 끝

2. 시작하고 쿨 걸기 (3~4)

- GA가 시작하자마자 쿨 Effect를 보스에게 걸어 둠(Commit) → 8초 동안 다시 못 씀

3. 기다리다가 순간에 깨어남 (5~8)

- 몽타주 재생 Task + 신호 대기 Task를 켜 둠 → Notify가 Strike 이벤트를 보내면 GA가 깨어나 점 12개 계산

4. 맞으면 (9~11)

- GA는 GE를 "건네기만" 함 → 플레이어 ASC가 숫자 · 태그를 바꾸고, GE에 적힌 Cue가 이펙트를 띄움

5. 끝 (12)

- 몽타주가 끝나면 Task가 알려 주고 GA가 End — 도중에 보스가 기절하면 Cancel로 Task까지 한 번에 정리

6 우리 프로젝트에선 무엇이 이 역할을 하나

GAS 용어	③ 보스 (Pattern 구조)	② AbilityData (플레이어 · 옛 보스)
ASC	BossMonsterController + PatternContext	AbilityRunner + 캐릭터 슬롯 목록
Attribute	MonsterStat	CharacterStat
GameplayTag	없음 — bool · HashSet · 직접 참조	ApplyStatusTag (상태 이름표만)
GameplayAbility	Pattern + PatternRunner	AbilityData (흐름 · 결과가 한 목록)
AbilityTask	AnimationAction · TimelineAction	RepeatEffect · DelayEffect (Effect로)
쿨다운 · 코스트	표의 쿨 (코스트 없음, 보스엔 불필요)	AbilityData.Cooldown · Cost
AnimNotify	타임라인 시그널	애니메이션 이벤트 (AbilityBeat)
GameplayEvent	FireEvent — 이름 대신 직접 가리킴	AbilityBeat → UseAbility — 직접 호출
Targeting	Placement (CirclePlacement)	Anchor · ScatterPattern · RadialBurst
GameplayEffect	없음 — StrikeEvent 안에 피해 코드	DealDamage · AreaStrike · ApplyStatModifier
GameplayCue	없음 — Event가 직접 이펙트 생성	SpawnVFX (결과와 같은 목록)

있음 (역할이 따로 있음) 비슷 (다른 것과 섞여 있음 · 직접 연결) 없음

한 줄씩 정리

1. ②는 GAS의 "결과" 쪽을 갖고 있다

- DealDamage · ApplyStatModifier · SpawnVFX = GameplayEffect · Cue와 같은 일 — 단, 흐름(Repeat · Delay)까지 같은 목록에 섞임

2. ③은 GAS의 "흐름" 쪽을 갖고 있다

- Pattern = Ability, Action = AbilityTask, Placement = Targeting — 단, 결과 · 연출 층이 없음

3. 그래서 둘을 합치면 GAS 모양이 된다

- ③에 결과 층(HitEffect)을 만들고, 거기서 ②의 결과 Effect를 살려 쓰는 것이 다음 단계

4. 태그는 지금 안 들여도 된다

- 쓸 곳은 둘(이벤트 이름 · 기절 같은 상태)뿐 — 이벤트 키 하나와 기존 상태 값으로 충분, 늘어나면 그때

GAS가 하는 일 = 흐름 · 결과 · 연출을 나눠서, 서로 이름으로만 부르게 하는 것